

Functions:

- A function is a group of statements made to execute them more than once in a program. A function has a name.
- Functions can compute a result value and can have parameters that serves as function inputs which may differ each time when function is executed.
- Functions are used *to reduce the size of code as it increases the code reusability* and *split a complex problem into multiple modules (functions) to improve manageability*.
- Sequential codes are easy for small scale programs. It becomes harder to keep track of details when code size exceeds.

Advantages:

- Modularity
- Abstraction
- Code reusability

Functions: Modularity, Abstraction & Reusability

Modularity

Breaking program into smaller, self-contained parts

Abstraction

Hiding the complex details, showing only what is necessary

Code Reusability

Using the same code multiple times without rewriting

- A function is a block of code which only runs when it is called.
- You can pass data, known as parameters, into a function.
- A function can or cannot return data as result.

Function Components:

- *Function signature*

Function Name

Function Arguments (optional)

- *Doc string*

- *Function Body*

- *Function return statement (optional)*

Syntax:

```
def function_name(arguments):  
    doc string  
    body  
    return statement
```

```
def my_function(fname, lname):  
    """  
        function to print the first and last name  
    """  
    print(fname + ' ' + lname)
```

```
my_function('Red', 'apple')  
Red apple  
my_function('Green', 'chilli')  
Green chilli
```

```
def my_function(country = "Pakistan"):  
    print("I am from " + country)
```

```
my_function()  
I am from Pakistan  
my_function("Karachi")  
I am from Karachi
```

```

def circle(r):
    """
        Returns the circumference of circle
    """
    return 2*3.14*r

print(circle.__doc__)

    Returns the circumference of circle

circle(7)
43.96
x = 5
circle(x)
31.400000000000002

```

Scope of Variables:

Scope of variable defines the region of program where a variable is recognized and accessible.

LEGB Rule:

When Python looks for a variable, it search in this order: Local, then Enclosing, then Global and then Built-in. If variable is not found in these scopes, a ***NameError*** is raised.

Built-in (Python)

Names pre-assigned in the built-in names model: open, range, SyntaxError...

Global (module)

Names assigned at the top-level of module file or declared global in a def within the file

Enclosing function locals

Names in the local scope of any and all enclosing functions, from inner to outer

Local (function)

Names defined in any way within a function and not declared global in that function

Local

```
def my_function():  
    local_var = 4          # This is local variable  
    print(local_var)
```

```
my_function()  
4
```

Global

```
global_var = 30
```

```
def modify_global():  
    global global_var  
    global_var = 19  
    print(global_var)
```

```
print(global_var)  
30  
modify_global()  
19  
print(global_var)  
19
```

Enclosing

```
def outer_function():  
    enclosing_var = 23  
    def inner_function():  
        print(enclosing_var)  
    inner_function()
```

```
outer_function()  
23
```

```
def outer():  
    x = 5  
    def inner():  
        x = 3  
        print("Inner x: ", x)  
    print("Outer x: ", x)  
    inner()
```

```
x = 10  
outer()  
Outer x: 5  
Inner x: 3  
print("Global x:", x)  
Global x: 10
```